PROJECT DOCUMENTATION

HR MANAGEMENT SYSTEM

A Windows Forms based Recruitment, Interview & Payroll Management Application

Technology Stack: C# .NET (WinForms) | ADO.NET | Microsoft SQL Server

2026

INDEX

CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1.	**ABSTRACT**	1
2.	**INTRODUCTION**	2
3.	**PROBLEM DEFINITION** • Existing System • Proposed System	3
4.	**SYSTEM REQUIREMENTS** • Software Requirements • Hardware Requirements	4
5.	**PROGRAM ENVIRONMENT** • Front End	5
6.	**SYSTEM DESIGN** • Data Flow Diagram	6
07.	**CONCLUSION AND FUTURE SCOPE**	7
08.	**FORMS AND REPORT**	10
09.	**BIBLIOGRAPHY**	13

1. ABSTRACT

The HR Management System is a desktop application built on the Microsoft .NET Framework (C#, Windows Forms) that automates the core day-to-day activities of a Human Resources department: publishing job openings, tracking candidates who apply against those openings, scheduling and recording the outcome of interviews, converting selected candidates into hired employees, and generating and exporting monthly salary reports and salary slips.

The system is organised as a layered application. A presentation layer (the "HR" Windows Forms project) is responsible only for user interaction. A business/entity layer (the "HrObjects" class library) defines the domain objects — Job, Candidate, Employee, Interview, Interview Schedule and Salary Report — that flow between the layers. A data access layer (the "DataAccess" class library, through its DataManager class) is the single point of contact with the underlying Microsoft SQL Server database, and it talks to the database exclusively through stored procedures, keeping SQL out of the user-interface code.

The application currently supports four end-to-end workflows: (1) posting a new job opening together with the required skills and years of experience for each skill; (2) registering candidates against a job opening and scheduling multi-round interviews with specific employees/interviewers; (3) recording interview outcomes — completed, selected, offer accepted — and converting an accepted candidate into a hired employee; and (4) maintaining employee salary records and exporting them as a yearly Excel workbook, a yearly PDF report, or an individual PDF salary slip.

This document describes the problem the system addresses, the requirements it was built against, the environment it runs in, its internal design (including the data flow through the system), representative source code, the testing approach used to validate it, and the screens a user interacts with.

2. INTRODUCTION

Human Resources teams in most organisations repeat the same cycle for every open position: a job is defined and advertised, candidates apply and are screened, shortlisted candidates are interviewed by one or more people, a decision is made and an offer is extended, and — once the candidate joins — the new employee enters the organisation's payroll cycle every month. When this cycle is managed through spreadsheets, e-mail threads and paper forms, information about a candidate's status, an interview's outcome or an employee's salary history becomes scattered and difficult to retrieve or audit.

The HR Management System was built to bring these four activities — job posting, candidate & interview management, employee records and salary reporting — into a single desktop application backed by one relational database. Because it is a Windows Forms MDI (Multiple Document Interface) application, an HR user works from one main window and opens the screen relevant to the task at hand (Job, Schedule Interview, Interview Status, or Salary Report) as a child window, in the same way a spreadsheet or word-processor application manages multiple open documents.

The remainder of this document explains why the system was needed (Problem Definition), what it was required to do (System Requirements), the platform it is built and deployed on (Program Environment), how it is structured internally (System Design), representative pieces of its implementation (Source Code), how it was verified (System Testing), what its screens look like (Screen Output), and the conclusions drawn from building it.

3. PROBLEM DEFINITION

3.1 Existing System

In the manual / semi-automated setup that this project replaces, HR-related information is typically maintained across separate, disconnected artefacts:

- Job openings are announced through e-mail or a shared document, with no single authoritative list of which jobs are currently open, closed, or on hold.
- Candidate applications (resume, contact details, qualifications, last employer) are collected in mail folders or a spreadsheet, making it hard to see, at a glance, all candidates who applied for a specific job.
- Interview scheduling is coordinated by e-mail or phone between the interviewer(s) and the candidate, with no central record of which interviewer is booked, at what time, for which candidate.
- Interview outcomes (completed / not completed, selected / rejected, offer accepted / declined) and the resulting joining date are recorded inconsistently, and there is no reliable link from "interview outcome" to "new employee record".
- Salary information — basic pay, allowances, bonus, deductions — is maintained in individual spreadsheets per employee or per month, which are error-prone to total, difficult to audit, and time-consuming to turn into a payslip or a yearly report.

The consequences of this fragmented approach are duplicated data entry, no single source of truth for a candidate's or an employee's status, difficulty producing consolidated reports, and a real risk of errors (e.g., double-booking an interviewer, or under/over-stating a salary component) going unnoticed.

3.2 Proposed System

The proposed HR Management System addresses the above problems by centralising all HR data in one SQL Server database and exposing it through a small set of task-focused screens:

- A Job Posting screen where a hiring manager's requisition is turned into a job record with department, candidate type and a list of required skills with the minimum years of experience for each.
- A Schedule Interview screen that lists all open jobs, shows the candidates who have applied for the job currently selected, allows a new candidate to be registered against that job, and allows an interview to be booked with a chosen interviewer for a specific start and end time.
- An Interview Status screen that lists every scheduled interview and lets HR progressively record whether the interview was completed, whether the candidate was selected, whether the offer was accepted, and — once accepted — the joining date, at which point the candidate is saved into the system as a hired employee.
- A Salary Report screen where a basic salary, allowances, bonus and deductions can be entered per employee per pay period; the net salary is calculated automatically, and the year's records can be exported to an Excel workbook or a consolidated PDF report, with an individual salary slip PDF available per employee per month.

Because every screen talks to the database only through the DataManager class and stored procedures, the same validated business rules apply everywhere the data is touched, and future screens (for example, an employee self-service portal) can reuse the same data access layer without duplicating logic.

4. SYSTEM REQUIREMENTS

4.1 Software Requirements

COMPONENT	SPECIFICATION
Operating System	Microsoft Windows 10 / 11 (development and client machine)
Programming Language	C# (.NET Framework)
Application Type	Windows Forms (WinForms), MDI (Multiple Document Interface)
Database Server	Microsoft SQL Server (Express or higher), database "HRDatabase"
Data Access	ADO.NET (System.Data.SqlClient) calling SQL stored procedures
IDE	Microsoft Visual Studio 2019 / 2022
Additional Libraries	Newtonsoft.Json, Owin (present in the project's package references)
Optional / Runtime	Microsoft Excel (for yearly XLS export via Excel Interop) — export still works as a PDF if Excel is not installed

4.2 Hardware Requirements

COMPONENT	SPECIFICATION
Processor	Dual-core 2 GHz or higher (Intel/AMD)
RAM	4 GB minimum (8 GB recommended for running SQL Server locally)
Hard Disk	500 MB free space for the application; additional space for the SQL Server database
Display	1024 × 768 resolution or higher
Other	Network connectivity between the client machine and the SQL Server instance, if not run locally

5. PROGRAM ENVIRONMENT

The solution (HR.sln) is composed of three Visual Studio projects that together form the running application:

- HR — the Windows Forms front-end project; the startup project (Program.cs) that launches the MDIParent1 main window.
- HrObjects — a class library holding the plain domain/entity classes (Job, Candidate, Employee, Interview, InterviewSchedule, InterviewDetails, SkillsWithExperience, SalaryReport) and the shared enumerations (Department, Skills, EmployeeType).
- DataAccess — a class library whose single DataManager class opens the SQL Server connection and executes stored procedures for every read/write operation used by the front end.

5.1 Front End

The front end is implemented entirely with Windows Forms controls — no web browser or HTML rendering is involved. The application opens with MDIParent1, a Multiple-Document-Interface parent form whose menu bar is the single entry point into every feature:

- Job → opens JobForm, used to publish a new job posting with its required skills.
- Interview → Schedule Interview → opens ScheuleInterview, used to view open jobs, register candidates and book interview slots.
- Interview → Interview Status → opens InterviewStatus, used to progress an interview from scheduled through to a hiring decision.
- Salary Report → opens SalaryReportForm, used to enter, view, and export employee salary data.

Each of these is opened as an independent Form (JobForm, ScheuleInterview, InterviewStatus and SalaryReportForm all derive from System.Windows.Forms.Form), so an HR user can have several task windows open side by side, arranged with the standard MDI Cascade / Tile Vertical / Tile Horizontal / Arrange Icons layout commands available from the Window menu. Standard WinForms controls are used throughout: TextBox and multiline TextBox for free text, ComboBox for single-value selection (skills, department, candidate type, employee, interviewer, year), ListView for tabular / multi-column read-only lists (skills added to a job, jobs, candidates, interviews), DataGridView for the editable salary-report grid, DateTimePicker for dates, RadioButton/GroupBox pairs for the progressive Yes/No decisions on the Interview Status screen, and NumericUpDown controls for currency fields on the Salary Report screen.

6. SYSTEM DESIGN

6.1 Layered / Architectural Design

The application follows a simple three-tier design that keeps user-interface code, domain objects and database access in separate projects, so that a change to how data is stored does not require changes to the screens, and vice-versa.

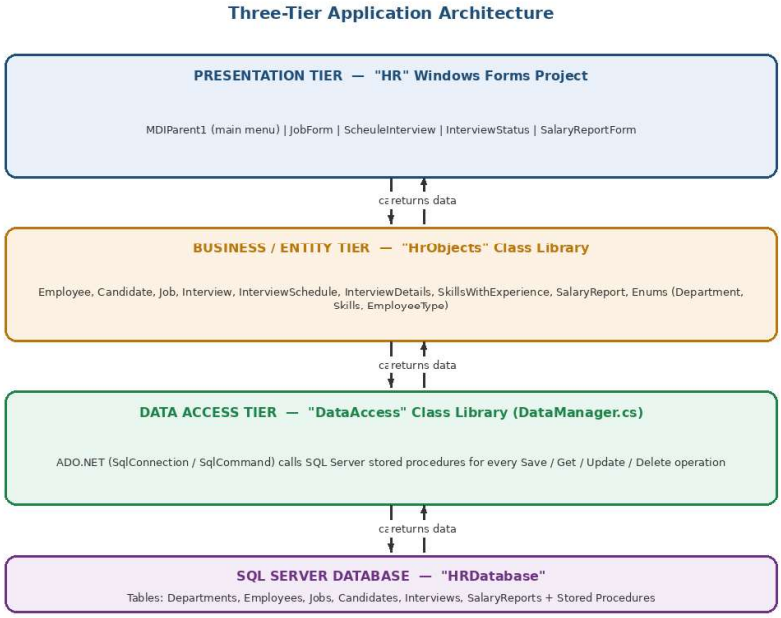


Figure 6.1 — Three-tier architecture of the HR Management System

6.2 Data Flow Diagram (DFD)

A Data Flow Diagram is used here to show how information moves between the people who use the system (external entities), the processes the system performs, and the data stores where information is kept at rest.

6.2.1 Level 0 DFD — Context Diagram

The context diagram treats the whole application as a single process and shows only the external entities that exchange data with it: the HR Administrator, the Hiring Manager, the Job Candidate and the Employee/Payroll staff.

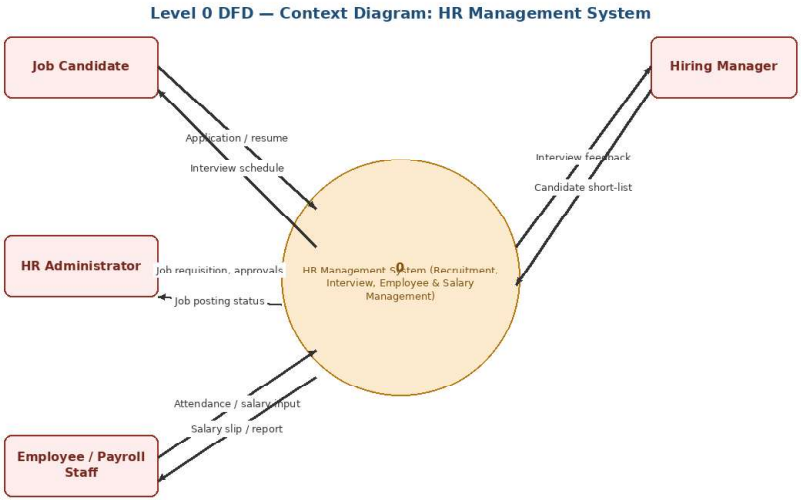


Figure 6.2 — Level 0 DFD (Context Diagram)

6.2.2 Level 1 DFD

The Level 1 diagram decomposes the single process into the four functional processes implemented by the system, each reading from and writing to its own data store(s) in the HRDatabase:

- 1.0 Manage Job Postings — writes to the Jobs store (uspJobPosting_Save) and is read by process 2.0 to show which jobs are open.
- 2.0 Manage Candidates & Interviews — writes to the Candidates store (uspCandidate_Save) and the Interviews store (uspInterviewDate_Save), and reads both to display candidates per job and interview status.
- 3.0 Manage Employees — reads/writes the Employees & Departments store (uspGet_AllEmployees, uspInsert_Employee, uspUpdate_Employee, uspDelete_Employee) and also receives newly-hired candidates from process 2.0 (uspSaveNewEmployee).
- 4.0 Manage Salary / Payroll Reports — reads the Employees store and writes/reads the SalaryReports store (uspSalaryReport_Save, uspSalaryReport_Get).

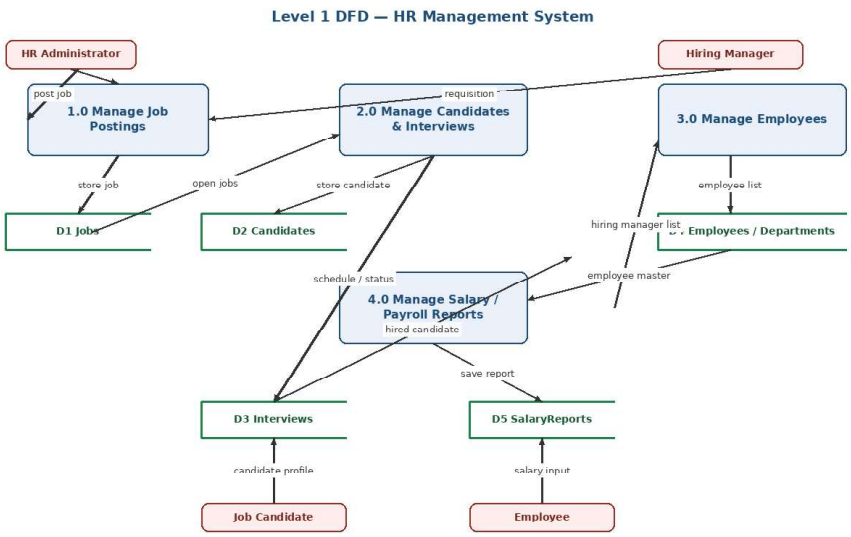


Figure 6.3 — Level 1 DFD of the HR Management System

6.3 Database Design (Key Tables)

The database, HRDatabase, holds one table per data store shown in the DFD. The principal tables and their columns, as created by the project's SQL scripts, are:

- Departments (DepartmentId PK, DepartmentName)
- Employees (EmployeeId PK, EmployeeFirstName, EmployeeLastName, DepartmentId FK → Departments, HireDate)
- Jobs (JobPostingId PK, JobTitle, JobDescription, JobPostingDate, JobDepartment, HiringManagerId, CandidateType, Status) with an associated required-skills table populated through a table-valued parameter
- Candidates (CandidateId PK, JobPostingId FK → Jobs, FirstName, LastName, Status, Qualifications, LastCompanyName, RecentGraduationDate)
- Interviews (InterviewId PK, JobPostingId FK, CandidateId FK, InterviewDate, InterviewStatus, related interviewer schedule rows)
- SalaryReports (SalaryReportId PK, EmployeeId FK → Employees, PayPeriod, BasicSalary, Allowances, Bonus, Deductions — unique per Employee + PayPeriod)

All reads and writes against these tables are performed exclusively via stored procedures (for example dbo.uspJobPosting_Save, dbo.uspGet_AllEmployees, dbo.uspInterview_Get, dbo.uspSalaryReport_Get), so that data validation and query logic live in the database layer rather than being duplicated across screens.

7. CONCLUSION AND FUTURE SCOPE

The HR Management System successfully consolidates the four core HR activities — job posting, candidate and interview management, employee record-keeping and salary reporting — into one Windows Forms application backed by a single SQL Server database. By separating the project into a presentation layer, an entity layer and a data access layer, and by pushing every read/write operation through stored procedures, the system keeps user-interface code free of SQL and gives every screen a single, consistent path to the data.

The workflow implemented — from posting a job, through registering and interviewing candidates, to converting an accepted candidate into an employee and finally into a salary report — mirrors the real recruitment-to-payroll lifecycle, and the export options (Excel, PDF report, PDF salary slip) make the salary data usable outside the application itself.

7.1 Limitations

- The connection string in DataManager currently points to a fixed local SQL Server instance, so deployment to another environment requires a configuration change rather than an application-setting change.
- There is no login/authentication screen in the current build, so access control would need to be added before the system is used by multiple HR staff with different permission levels.
- The yearly Excel export currently relies on Microsoft Excel being installed on the client machine (Excel Interop); the PDF export path does not have this dependency.

7.2 Future Scope

- Add user authentication and role-based access (e.g., HR Administrator vs. Hiring Manager vs. read-only Employee) so that not every screen is available to every user.
- Move the connection string and other environment-specific values into an external configuration file (App.config) so the same build can target different SQL Server instances.
- Add e-mail notifications when a job is posted, an interview is scheduled, or an offer is extended, to reduce the manual coordination that still happens outside the system.
- Extend the Employees screen with full add/update/delete UI (the DataManager methods already exist) and add search/filter capability to the Jobs, Candidates and Interviews lists as the volume of data grows.
- Migrate the desktop client to a web or cross-platform front end so HR staff and hiring managers can use the system without installing anything locally.

Overall, the project demonstrates a complete, working example of a layered desktop line-of-business application, and the groundwork it lays — a clean entity model, a stored-procedure-based data access layer, and task-oriented screens — provides a solid base for the enhancements listed above.

8. FORMS AND REPORT

This chapter brings together the data-entry forms and the reporting screen of the application. The forms below are representative layouts of the application's Windows Forms screens, reconstructed from the project's form code to illustrate each screen's controls and typical content; they are shown in the order a user moves through the recruitment-to-payroll workflow, ending with the salary report screen and its export options.

8.1 Main Menu (MDIParent1)

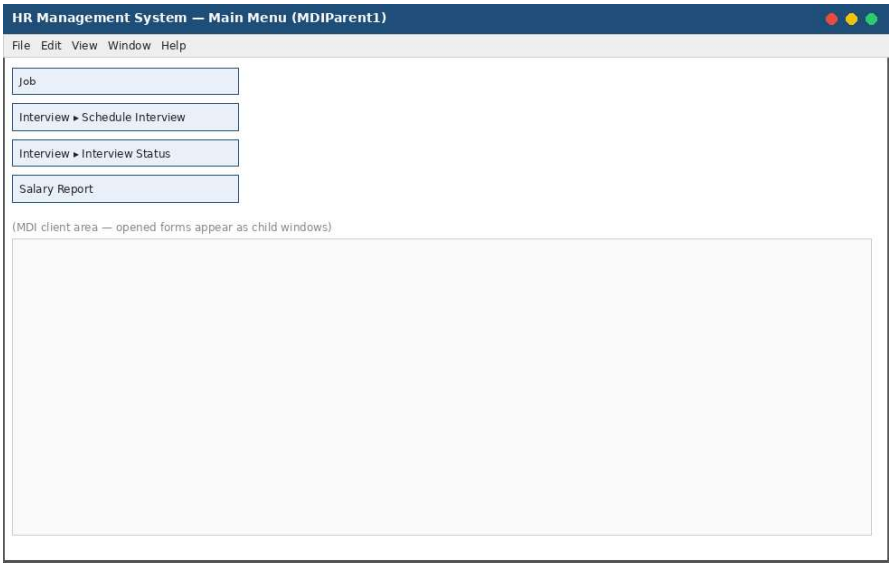


Figure 8.1 — Main MDI window with the Job, Interview and Salary Report menus

8.2 Job Posting Form (JobForm)

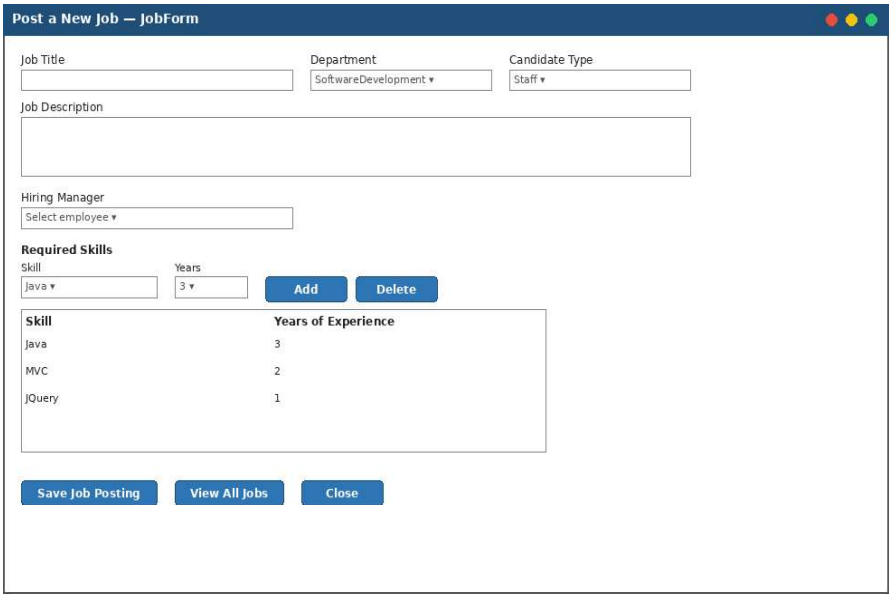


Figure 8.2 — Creating a job posting with required skills and years of experience

8.3 Schedule Interview Form

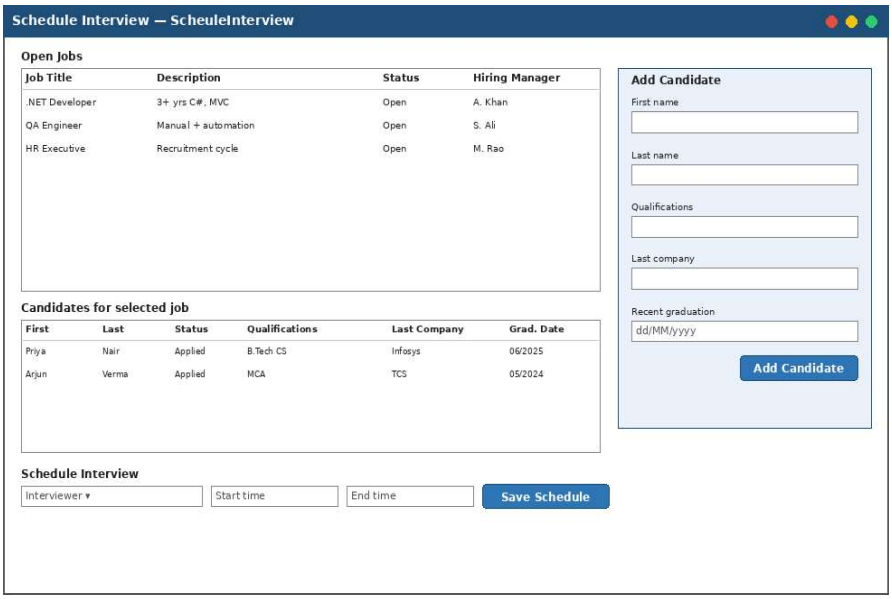


Figure 8.3 — Viewing open jobs and their candidates, adding a candidate, and booking an interview

8.4 Interview Status Form

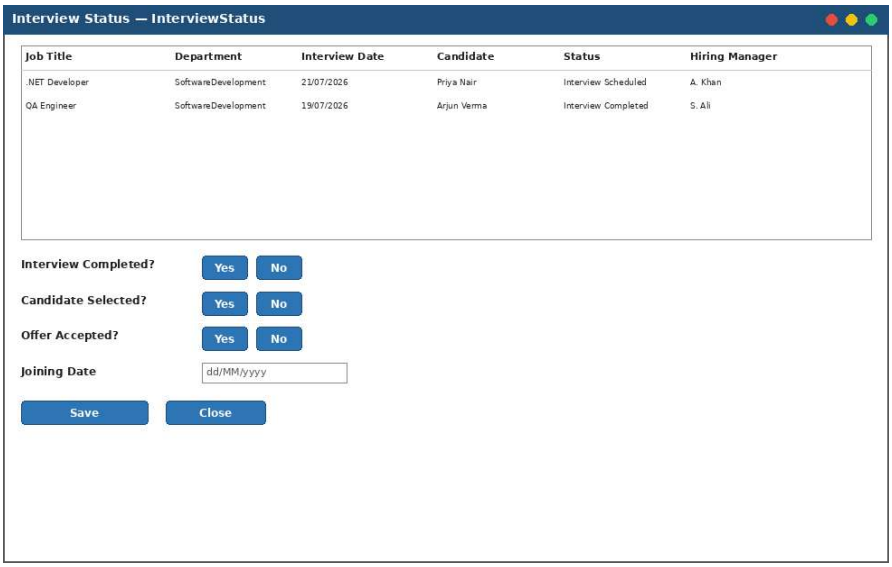


Figure 8.4 — Progressing an interview from scheduled to a hiring decision

8.5 Salary Report

The Salary Report screen doubles as the system's reporting screen: it lists every saved salary record in a grid and can turn the records for a chosen year into a yearly Excel workbook or a yearly PDF report, or turn a single selected record into a per-employee PDF salary slip.

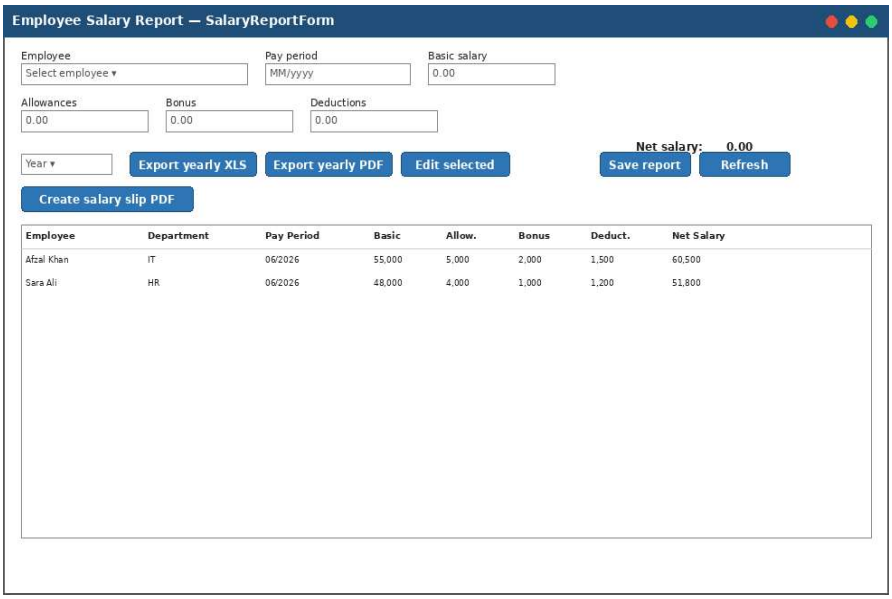


Figure 8.5 — Entering, viewing and exporting employee salary reports

9. BIBLIOGRAPHY

The following books, official documentation and online references were used while designing, developing and documenting this project.

9.1 Books

Pressman, Roger S., and Bruce R. Maxim. Software Engineering: A Practitioner's Approach. McGraw-Hill Education.

Sommerville, Ian. Software Engineering. Pearson Education.

Troelsen, Andrew, and Philip Japikse. Pro C# with .NET. Apress.

Elmasri, Ramez, and Shamkant B. Navathe. Fundamentals of Database Systems. Pearson Education.

9.2 Websites / Online Documentation

Microsoft. "C# documentation." Microsoft Learn — https://learn.microsoft.com/dotnet/csharp/

Microsoft. "Windows Forms documentation." Microsoft Learn — https://learn.microsoft.com/dotnet/desktop/winforms/

Microsoft. "ADO.NET overview." Microsoft Learn — https://learn.microsoft.com/dotnet/framework/data/adonet/

Microsoft. "Transact-SQL (T-SQL) reference." Microsoft Learn — https://learn.microsoft.com/sql/t-sql/

Microsoft. "SQL Server documentation." Microsoft Learn — https://learn.microsoft.com/sql/sql-server/

Microsoft. "Stored procedures (Database Engine)." Microsoft Learn — https://learn.microsoft.com/sql/relational-databases/stored-procedures/